

五子棋控制台版

基于现代 C++ 的面向对象游戏开发

项目名称：五子棋控制台版小游戏

视频网址：

https://www.bilibili.com/video/BV1TxJP66EuZ/?vd_source=0ec96477972a3030c5174c9628a82883#reply116748284073341

一、项目概述

1.1 功能说明

本项目实现了一个五子棋控制台小游戏。游戏采用双人对战模式，在 15×15 的标准棋盘上进行。玩家通过输入坐标来落子，先形成五子连珠的一方获胜。

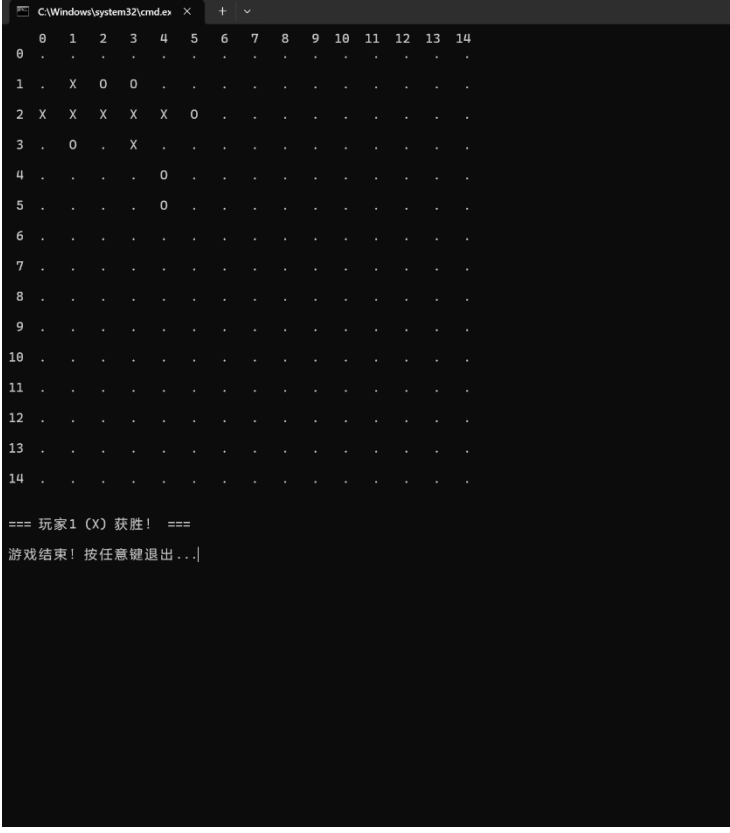
核心功能包括：

- 15×15 标准五子棋棋盘
- 双人对战，黑棋(X)先手，白棋(O)后手
- 玩家输入 xy 坐标落子
- 自动检测五子连珠并宣布获胜者
- 棋盘下满自动判定平局

1.2 操作方式

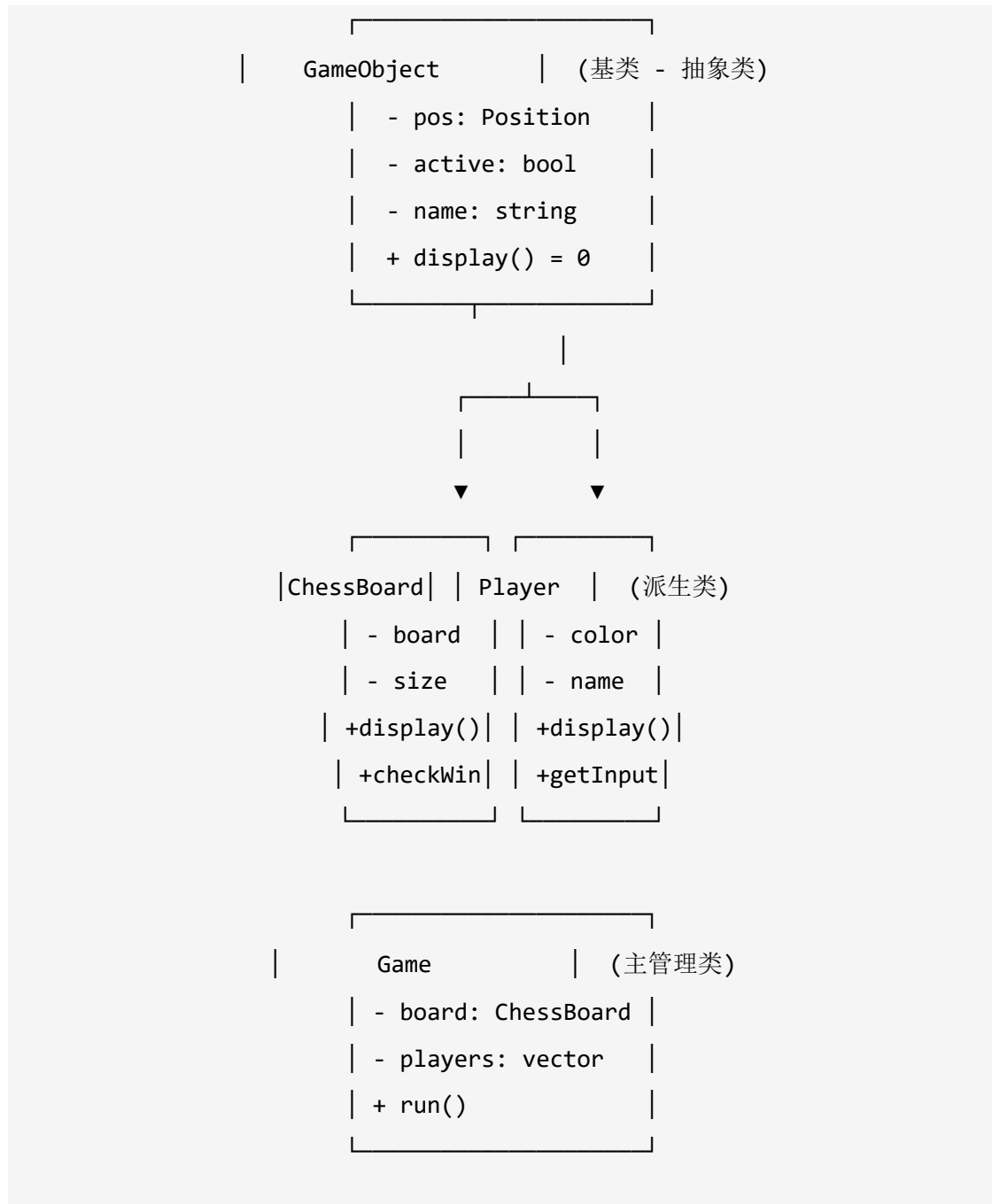
操作	说明
启动游戏	运行 gomoku.exe
输入 x y	在 (x,y) 位置落子（如：7 7）
自动判断	系统自动检测五子连珠

1.3 运行截图



二、 OOP 设计详解

2.1 类继承图



2.2 类的职责说明

GameObject (基类)：定义所有游戏对象的通用接口。包含纯虚函数 `display()`，强制派生类实现自己的显示逻辑。使用 `protected` 保护成员变量，体现封装性。

ChessBoard (派生类)：管理 15×15 棋盘，使用 `vector<vector<PieceColor>>` 存储棋子状态。重写 `display()` 显示棋盘，实现落子、五子连珠判断、平局检测等核心功能。

Player (派生类)：管理玩家信息和棋子颜色。重写 `display()` 显示棋子符号，处理坐标输入和输入验证。

Game (主管理类)：游戏主控制器，管理游戏循环、玩家切换、游戏状态等。负责所有游戏对象的生命周期管理。

2.3 虚函数与多态

基类 `GameObject` 定义了纯虚函数 `display()`，`ChessBoard` 和 `Player` 分别重写该函数。通过基类指针调用 `display()` 时，实际执行的是派生类的版本，实现了运行时多态。

```
// 多态示例
GameObject* obj1 = new ChessBoard();
GameObject* obj2 = new Player("玩家 1", PieceColor::Black);
obj1->display(); // 调用 ChessBoard::display()
obj2->display(); // 调用 Player::display()
```

2.4 OOP vs 非 OOP 对比

如果不使用面向对象编程，代码将采用面向过程的 C 风格写法：

- **数据与操作分离**：棋盘数据、玩家数据分散在全局变量中，函数各自操作这些数据
- **难以扩展**：添加新功能需要修改多处代码
- **代码复用性差**：相似功能需要重复编写

使用 OOP 带来的好处：

- **可维护性**：数据和操作封装在一起，修改影响范围小
- **可扩展性**：添加新类型只需继承基类，重写虚函数
- **代码复用**：基类定义通用接口，派生类共享代码
- **多态性**：统一接口处理不同类型对象

三、现代 C++ 特性运用分析

3.1 vector 管理棋盘

使用 `vector<vector<PieceColor>>` 管理二维棋盘，代替传统的 C 风格二维数组。

```
// 现代写法：使用 vector
vector<vector<PieceColor>> board;
board.resize(size, vector<PieceColor>(size, PieceColor::Empty));

// 传统 C 风格写法：
PieceColor** board = new PieceColor*[size];
for (int i = 0; i < size; i++) {
    board[i] = new PieceColor[size];
    memset(board[i], 0, sizeof(PieceColor) * size);
}
// 需要手动 delete[] 释放内存
```

优势： `vector` 自动管理内存，无需手动释放；提供边界检查；支持动态调整大小。

3.2 范围 for 循环与 auto

使用范围 for 和 auto 遍历容器，代码简洁直观。

```
// 现代写法：范围 for + auto
for (const auto& row : board) {
    for (auto cell : row) {
        if (cell == PieceColor::Empty) return false;
    }
}
```

```
// 传统写法：使用索引
for (int i = 0; i < size; i++) {
    for (int j = 0; j < size; j++) {
        if (board[i][j] == PieceColor::Empty) return false;
    }
}
```

优势：代码更简洁，意图更明确；避免索引越界错误；`auto` 自动推导类型，减少重复书写。

3.3 enum class 类型安全枚举

```
// 现代写法：enum class
enum class PieceColor {
    Empty, Black, White
};
// 使用时：PieceColor::Black

// 传统写法：普通枚举
enum PieceColor { EMPTY, BLACK, WHITE };
// 全局可见，容易命名冲突
```

优势：`enum class` 提供类型安全的作用域枚举，不会污染全局命名空间，不能隐式转换为整数。

3.4 其他现代 C++ 特性

- **nullptr**：代替 NULL，类型安全
- **override**：显式标记重写函数，编译器检查
- **default**：显式使用默认构造函数
- **列表初始化**：`Position(int x = 0, int y = 0) : x(x), y(y) {}`
- **std::string**：代替 `char[]`，自动管理内存

四、核心难点与解决方案

4.1 五子连珠判断算法

问题描述：最初实现五子连珠判断时，只检查了从落子点向一个方向延伸，忽略了反方向，导致只能检测到单向的连珠。

解决方案：对四个方向（水平、垂直、主对角线、副对角线），分别从落子点向正反两个方向延伸检查，将两个方向的连子数相加。

```
bool checkWin(int x, int y, PieceColor color) const {  
    int dx[] = {1, 0, 1, 1}; // 四个方向  
    int dy[] = {0, 1, 1, -1};  
  
    for (int dir = 0; dir < 4; dir++) {  
        int count = 1; // 当前棋子  
        // 正方向延伸  
        for (int step = 1; step < 5; step++) {  
            int nx = x + dx[dir] * step;  
            int ny = y + dy[dir] * step;  
            if (超出边界或颜色不同) break;  
            count++;  
        }  
  
        // 反方向延伸  
        for (int step = 1; step < 5; step++) {  
            int nx = x - dx[dir] * step;  
            int ny = y - dy[dir] * step;  
            if (超出边界或颜色不同) break;  
            count++;  
        }  
  
        if (count >= 5) return true;  
    }  
    return false;  
}
```

4.2 控制台编码兼容性

问题描述：中文界面在部分控制台显示为乱码。

解决方案：使用 `SetConsoleOutputCP(65001)` 设置控制台编码为 UTF-8，确保中文正常显示。

五、编译与运行说明

项目	要求
操作系统	Windows 10/11
编译器	MinGW GCC 6.3.0+ 或 Visual Studio 2017+
C++标准	C++11 或更高版本

编译命令：

```
g++ -std=c++11 -o gomoku.exe game.cpp
```

运行方式：

```
.\gomoku.exe
```


六、总结与心得

6.1 项目收获

- **深入理解 OOP 三大特性**：通过实际项目运用封装、继承、多态，加深了对面向对象编程的理解
- **掌握现代 C++ 编程风格**：学会使用 vector、string、auto、范围 for、enum class 等现代特性
- **提升算法设计能力**：设计和调试五子连珠判断算法，锻炼了逻辑思维能力
- **增强调试能力**：解决编码兼容性问题，提升了问题定位和解决能力

6.2 不足与改进方向

- **AI 对战功能**：可添加简单 AI，实现人机对战
- **悔棋功能**：支持撤销上一步操作
- **图形界面**：可改为图形界面，提升用户体验
- **网络对战**：支持局域网或互联网对战

6.3 心得体会

这次项目让我深刻体会到，好的程序设计不仅仅是让代码能运行，更重要的是让代码好维护、易扩展。面向对象设计和现代 C++ 特性正是实现这一目标的有力工具。在开发过程中，我遇到了不少挑战，比如五子连珠判断算法的设计、控制台编码兼容性等。通过查阅资料、反复调试，最终都找到了解决方案。这个过程虽然辛苦，但收获颇丰。

通过这个项目，我不仅巩固了课堂上学到的知识，还学会了如何将理论知识应用到实际开发中。我相信这些经验对我未来的学习和工作都会有很大帮助。

附录：完整源代码

完整源代码文件：game.cpp

GitHub 仓库：https://github.com/fiZZle0710/cpp_classreport.git